

— Work in progress —

Reinforcement Learning: Deep Q-Learning for Dominion

Contents

Reinforcement Learning

Reinforcement learning is a subtype of machine learning where an agent learns to optimize its behavior in a given environment through learning by trial and error. The agent learns from its experience what it should and shouldn't do within the state that it currently occupies.

Historical Background

blah blah blah

Sequential Decision Making

Given states S_t at time t , future states S_{t+1} , the rewards R_t for state transitions, and the actions A_t that map S_t into S_{t+1} , the reinforcement agent seeks to maximize its return

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where $\gamma \in [0, 1]$ is a discount factor for future returns.

=====

=====

Markov Decision Processes

A Markov decision process (MDP) is a 4-tuple defined as

$$(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P} = P(s' | s, a)$ is the probability of transitioning from state s to s' through action a , and $\mathcal{R}(s, a, s')$ is the reward for the transition from state s to s' through action a . Generally the reward is written as

$$r(s, a, s') = \mathbb{E}[R_{t+1} | S_t = s, A_t = a, S_{t+1} = s']$$

MDPs have the property where the probability of an event happening in the present (s, a) is independent of the process's history:

$$P(S_{t_1} | S_t, A_t) = P(S_{t+1} | S_t, A_t, S_{t-1}, A_{t-1}, \dots) .$$

Policies, Returns, and Value Functions

Following a deterministic policy, the agent's behavior is defined by

$$a = \pi(s) \in \mathcal{A}$$

where $\pi(s)$ is a policy mapping each state s to an action a . Acting on a deterministic policy, the agent picks the action that gives it the greatest reward. A stochastic policy $\pi(a | s)$ maps states to distributions over actions.

The state-value function

$$V^\pi(s) = \mathbb{E}[G_t | S_t = s] \quad (1) \% \% \text{MAGIT_P_ARSER_P_ROTECT}$$

gives the expected return for the agent if it starts in state s and follows policy π in all future steps. The state-action value function

$$Q^\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

gives the expected return for the agent if it starts in state s and follows action a and then policy π forwards. The optimal policy is one in which the action-value function is maximized:

$$\pi_*(s) \in \operatorname{argmax}_a Q_*(s, a)$$

Bellman Equations

The Bellman Expectation Equations are recursive equations used for policy evaluation of fixed policies. Starting with the [state-value equation](#) and substituting in the definition of G_t , we have

$$V^\pi(s) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right].$$

Separating the first term from the rest of the summation, factoring out a γ from the remaining sum, and reindexing, we have

$$V^\pi(s) = \mathbb{E} \left[\gamma^0 R_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k R_{t+k+2} | S_t = s \right].$$

And applying linearity of expectation, this becomes

$$\begin{aligned} V^\pi(s) &= \mathbb{E}[\gamma^0 R_{t+1} | S_t = s] + \gamma \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+2} | S_t = s \right] \\ V^\pi(s) &= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s]. \end{aligned}$$

This equation says that the value of the current state under policy π is the sum of the expected immediate reward of transitioning from state s to s' and the discounted expected future return G_{t+1} starting from the next time step. The expected immediate reward for being in the current state is computed as the summation over all possible actions and all possible successor states while multiplying together the probability that the agent will take action a ($\pi(a|s)$) while in state s , the probability that the environment will transition to state s' ($P(s'|s, a)$), and the reward for transitioning to state s' through action a ($r(s, a, s')$):

$$\mathbb{E}[R_{t+1} \mid S_t = s] = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} P(s'|s, a) r(s, a, s') .$$

The expected future return for being in state s is found through summing over all actions and all states s' and multiplying $\pi(a|s)$, $P(s'|s, a)$, and the state-value of the next state ($V^\pi(s')$):

$$\mathbb{E}[G_{t+1} \mid S_t = s] = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} P(s'|s, a) V^\pi(s') .$$

Put together, we have

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \left(\pi(a|s) \sum_{s' \in \mathcal{S}} [P(s, a, s') (r(s, a, s') + \gamma V^\pi(s'))] \right) .$$

The Q – *value* of a state-action pair is defined as the reward received for taking action a in state s plus the discounted expected value of being in the new state:

$$Q(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^\pi(s') . \quad (2) \% \% \text{MAGIT_PARSER_PROTECT}$$

If the agent follows an optimal policy π^* , then the value of being in some state is equal to the return that the agent gets from taking the optimal action within that state corresponding to the largest possible Q -value out of all of the agent's options:

$$V^*(s) = \max_a Q^*(s, a) .$$

The [state-action value equation](#) is then

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) \max_a Q^*(s, a) .$$

Temporal-Difference Learning

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Q-Learning

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Exploration and ϵ -Greedy Action Selection

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Tabular Q-Learning

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Example of Tabular Q-Learning — A Simple Deckbuilder

Strengths and Limitations of the Tabular Q-learning

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Deep Q-Learning

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Neural Networks

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

State Value and Action Value Approximation Using Neural Networks

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Deep Q-Learning for Dominion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.